Literature Review Pack: "Automated Mutation-Based Compiler Code Optimization" (BCSE307P, Review 1)

Our prototype is best described as a small educational reimplementation of established ideas: classic dataflow optimizations from the Dragon Book, a mutation-and-cost search loop like STOKE and genetic improvement, and test-based correctness checking like differential testing. It is not a novel optimization technique, and the literature does not support claiming novelty. What it offers is transparency: operator-level mutations over TAC, with every candidate logged, validated and scored, on a language small enough to inspect by hand.

TL;DR

- **Positioning:** Our pipeline (naive TAC → single-site mutations → differential-test validation → weighted cost → greedy acceptance) is a scaled-down, deterministic version of the "propose–test–score" loop used by STOKE (Schkufza et al., 2013) and genetic improvement (Petke et al., 2018). The operators themselves (M1–M9) are textbook optimizations (Aho et al., 2006). Present it as an educational prototype, not a contribution.
- **Eight core verified sources** cover all eight topics: Aho et al. (2006), Schkufza et al. (2013), Petke et al. (2018), Kulkarni et al. (2006), Tate et al. (2009), Lopes et al. (2021), Yang et al. (2011) and Cummins et al. (2022). Eight further verified sources are listed as optional backups.
- **Key honest message for the panel:** 100% of candidates passed validation because our operators are designed to be sound. Validation is a safety net, not the thing that guarantees correctness. Our fuzzing bug (19/300 programs miscompiled before the fix) shows the same lesson as Csmith and Alive2: tests miss rare cases such as division by zero, and only formal checking (planned future work) gives guarantees.

Per-Source Cards (8 core sources)

1. Aho, Lam, Sethi & Ullman (2006) — *Compilers: Principles, Techniques, and Tools* (2nd ed.)

- **APA:** Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2nd ed.). Addison-Wesley. ISBN 978-0-321-48681-3. abebooks ↗

bibsonomy
↗

- **What it does:** It is the standard compiler textbook (the "Dragon Book"). pearson
↗
Wikipedia
↗
It covers lexing, parsing, syntax-directed translation, intermediate code (including three-address code), and machine-independent optimizations based on data-flow analysis.
- **Concept relevant to us:** TAC as an intermediate representation, plus the classic local optimizations: constant folding, constant/copy propagation, common-subexpression elimination, dead-code elimination via liveness, strength reduction, and algebraic simplification.
- **How it motivates our methodology:** It is the source of our front end (lexer → recursive-descent parser → AST → TAC) and of operators M1–M7 and M9. Our M5 (backward liveness) and M6 (CSE) are textbook basic-block techniques. *Verification note:* bibliographic data were verified via the Pearson catalogue (ISBN 9780321486813, 2006). pearson
↗
Check chapter and page numbers against your own copy before citing specific sections.

2. Schkufza, Sharma & Aiken (2013) — Stochastic Superoptimization (STOKE)

- **APA:** Schkufza, E., Sharma, R., & Aiken, A. (2013). Stochastic superoptimization. In *Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '13)* (pp. 305–316). ACM. https://doi.org/10.1145/2451116.2451150 acm
↗
uni-trier
↗
- **What it does:** It treats loop-free x86-64 superoptimization as a stochastic search. A cost function combines a correctness term and a performance term, and a Markov Chain Monte Carlo sampler explores the program space. According to the paper's abstract, starting from binaries compiled by `llvm -O0` for 64-bit x86, STOKE produced programs that match or outperform the code produced by gcc with full optimizations enabled and, in some cases, expert handwritten assembly.
- **Concept relevant to us:** It uses mutation moves called Opcode, Operand, Swap and Instruction. Correctness is approximated by testcases during search (by default 32 per target) and then confirmed by a sound SMT-based validator (STP, bit-vector theory). If validation fails, the counterexample becomes a new testcase. Performance is a static sum of instruction latencies. arxiv
↗
- **How it motivates our methodology:** This is the closest prior work to ours, and we must acknowledge it explicitly. Our mapping is direct: single-site mutations correspond to STOKE's moves, our 25-input differential test corresponds to its testcase-based correctness term, our cost model to its `perf(·)`, and our cycle model to its static latency estimate. The differences

matter: STOKE's moves are random and often produce incorrect code, so it needs a graded "almost correct" cost and a formal validator. arxiv ↗

Our operators are semantics-preserving by design, so our candidates are almost always correct (100% in the dataset run). We use greedy acceptance rather than MCMC, and we have no formal validator. STOKE's swap move is the analogue of our M8 (Instruction Reordering).

3. Petke, Haraldsson, Harman, Langdon, White & Woodward (2018) — Genetic Improvement of Software: A Comprehensive Survey

- **APA:** Petke, J., Haraldsson, S. O., Harman, M., Langdon, W. B., White, D. R., & Woodward, J. R. (2018). Genetic improvement of software: A comprehensive survey. *IEEE Transactions on Evolutionary Computation, 22*(3), 415–432. https://doi.org/10.1109/TEVC.2017.2693219 stir ↗
- **What it does:** It surveys genetic improvement (GI), defined as "automated search to find improved versions of existing software," focusing on core papers from 1995–2015. Of the core empirical studies, 96% use evolutionary algorithms, mostly genetic programming. The authors report "a significant upsurge in activity since 2012."
- **Concept relevant to us:** Its general recipe is to mutate existing code, evaluate candidates against a test suite (functional correctness) and a fitness function (non-functional properties such as speed or size), and keep improvements.
- **How it motivates our methodology:** It provides the software-engineering framing for our "mutation + test-based validation + fitness" loop. Our weighted cost (0.4·I + 0.3·A + 0.2·T + 0.1·C) is a multi-objective fitness collapsed into a single score. GI's reliance on tests as the correctness oracle explains why we treat validation as empirical evidence rather than proof.

4. Kulkarni, Whalley, Tyson & Davidson (2006) — Exhaustive Optimization Phase Order Space Exploration

- **APA:** Kulkarni, P. A., Whalley, D. B., Tyson, G. S., & Davidson, J. W. (2006). Exhaustive optimization phase order space exploration. In *Proceedings of the International Symposium on Code Generation and Optimization (CGO '06)* (pp. 306–318). IEEE. https://doi.org/10.1109/CGO.2006.15 dblp ↗ acm ↗
- **What it does:** It tackles the long-standing *phase-ordering problem*: optimizations interact, so applying them in different orders gives different code. It shows that the space of distinct phase orderings can be enumerated exhaustively for most functions by pruning orderings that lead to identical code. The extended TACO 2009 version reports this "in a reasonable amount of time for most of the functions in our benchmark suite." acm ↗

- **Concept relevant to us:** Optimization phases enable and disable one another, and the order in which they are applied matters. acm ↗
- **How it motivates our methodology:** Our greedy search does not fix an order. Each iteration tries every operator at every site and accepts the best one, so it picks a single path through the phase-order space. That is why it can get stuck in local optima (a stated limitation), and why beam search or simulated annealing is planned. Our deduplication of identical candidates is a small-scale version of Kulkarni et al.'s pruning of identical function instances. *Verification note:* venue, pages and authors were verified via dblp (CGO 2006: 306–318). dblp ↗
  The DOI was taken from a citing ACM reference, and the content summary relies partly on the TACO 2009 follow-up abstract.

5. Tate, Stepp, Tatlock & Lerner (2009) — Equality Saturation: A New Approach to Optimization

- **APA:** Tate, R., Stepp, M., Tatlock, Z., & Lerner, S. (2009). Equality saturation: A new approach to optimization. In *Proceedings of the 36th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '09)* (pp. 264–276). ACM. https://doi.org/10.1145/1480881.1480915 acm ↗
- **What it does:** Instead of destructive, sequential passes, optimizations add *equalities* to a shared IR (E-PEGs). The engine saturates the IR with equivalent versions, then a global heuristic picks the best one. This "obviates the need to worry about optimization ordering" and can also perform translation validation. It was implemented in Peggy, a Java bytecode optimizer. ucsd ↗ iitd ↗
- **Concept relevant to us:** It offers a non-destructive alternative to greedy rewriting and treats extraction of the best program as a cost-based choice among equivalent programs.
- **How it motivates our methodology:** It is our main *contrast* point. Greedy acceptance commits to one rewrite at a time, while equality saturation keeps all rewrites at once and so avoids local optima from ordering. Our cost function plays the role of its extraction heuristic. An e-graph back-end (e.g., using the egg library) is a candidate future extension, not something we implement.

6. Lopes, Lee, Hur, Liu & Regehr (2021) — Alive2: Bounded Translation Validation for LLVM

- **APA:** Lopes, N. P., Lee, J., Hur, C.-K., Liu, Z., & Regehr, J. (2021). Alive2: Bounded translation validation for LLVM. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI '21)* (pp. 65–79). ACM.

https://doi.org/10.1145/3453483.3454030 sigplan + 2
↗

- **What it does:** It is a fully automatic, SMT-based, *bounded* translation validator for LLVM IR. It checks that optimized IR refines the original, is designed to avoid false alarms, and requires no changes to LLVM. sigplan
↗
Running it over LLVM's unit tests found 47 new bugs (28 fixed at publication) and led to eight patches to the LLVM Language Reference. sigplan
↗
- **Concept relevant to us:** Translation validation, in the line of Pnueli et al. (1998) and Necula (2000), checks each individual transformation result rather than proving the optimizer correct once and for all. berkeley
↗
It reasons about undefined behaviour, such as division by zero, symbolically.
- **How it motivates our methodology:** Our validator is a lightweight, *test-based* analogue: it checks each candidate against the original, comparing printed output and error status. Alive2 shows what we lack, namely proof over *all* inputs. Our fuzzing bug, where a possibly-trapping division was deleted or moved and none of the 25 inputs triggered division by zero, is exactly the class of bug symbolic validation catches. This motivates SMT-based equivalence checking as future work.

7. Yang, Chen, Eide & Regehr (2011) — Finding and Understanding Bugs in C Compilers (Csmith)

- **APA:** Yang, X., Chen, Y., Eide, E., & Regehr, J. (2011). Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '11)* (pp. 283–294). ACM. https://doi.org/10.1145/1993498.1993532 uni-trier
↗
arxiv
↗
- **What it does:** It introduces Csmith, a random C program generator for *randomized differential testing* of compilers. Over three years the authors reported more than 325 previously unknown bugs in GCC, LLVM and other compilers. utah
↗
utah
↗
The paper builds on McKeeman's (1998) differential testing.
- **Concept relevant to us:** Random program generation combined with comparing outputs across implementations, used as a bug-finding oracle.
- **How it motivates our methodology:** It justifies two parts of our pipeline. The first is per-candidate differential testing (original TAC vs. mutated TAC, plus a final check against an independent reference AST interpreter). The second is our fuzzing campaign: 300 random programs exposed 19 miscompilations, and after the soundness fix 1,300 random programs

(~20k accepted mutations) showed 0 incorrect results. Like Csmith, our fuzzing finds bugs but cannot prove their absence.

8. Cummins et al. (2022) — CompilerGym

- **APA:** Cummins, C., Wasti, B., Guo, J., Cui, B., Ansel, J., Gomez, S., Jain, S., Liu, J., Teytaud, O., Steiner, B., Tian, Y., & Leather, H. (2022). CompilerGym: Robust, performant compiler optimization environments for AI research. In *Proceedings of the 20th IEEE/ACM International Symposium on Code Generation and Optimization (CGO '22)* (pp. 92–105). IEEE. https://doi.org/10.1109/CGO53902.2022.9741258 acm ↗ acm ↗
- **What it does:** It exposes real compiler optimization tasks, such as LLVM phase ordering over a discrete choice of 124 passes, as OpenAI-Gym environments. It bundles millions of benchmark programs, including AnghaBench (1,041,333), cBench (23), CHStone (12), MiBench (40), NPB (122), GitHub (49,738) and the Csmith and llvm-stress generators. Excluding generators, it contains 1,145,499 benchmarks. For runnable benchmarks it applies differential testing to detect miscompilations. arxiv ↗
- **Concept relevant to us:** Standardized benchmark datasets and reproducible evaluation for optimization search, with differential testing as a built-in semantic check.
- **How it motivates our methodology:** It is the reference point for our dataset design: P01–P15 in six categories, seed 2026, with 62 expected-output tests. It also shows honestly how small our benchmark is (15 straight-line programs vs. over 1 million real programs). Scaling to CompilerGym or cBench programs requires control flow and LLVM IR, which is future work.

Optional backup sources (verified; cite if the panel asks)

Source	Verified details	One-line relevance
Massalin (1987)	ASPLOS II, pp. 122–126, doi 10.1145/36206.36194	Coined "superoptimizer"; exhaustive search for the shortest program, with a probabilistic test to make it practical acm ↗ Wikipedia ↗
Bansal & Aiken (2006)	ASPLOS XII, pp. 394–403, doi 10.1145/1168857.1168906	Automatically learned peephole rules via brute-force superoptimization; STOKE follows its testcase-then-verify

Source	Verified details	One-line relevance
		architecture arxiv + 2 ↗
Cooper, Schielke & Subramanian (1999)	LCTES '99, pp. 1–9, doi 10.1145/314403.314414	Genetic algorithm searching optimization *sequences* for smaller code acm ↗ acm ↗
Ashouri et al. (2018)	ACM Computing Surveys 51(5), Art. 96, pp. 1–42, doi 10.1145/3197978	Survey of ML autotuning covering optimization selection and phase ordering acm ↗
Willsey et al. (2021)	PACMPL 5(POPL), Art. 23, doi 10.1145/3434304	egg e-graph library ("rebuilding", e-class analyses) that makes equality saturation practical nsf ↗ sigplan ↗
Necula (2000)	PLDI '00, pp. 83–94, doi 10.1145/349299.349314	Translation validation for GCC that checks IR before and after each pass acm ↗ semanticscholar ↗
Pnueli, Siegel & Singerman (1998)	TACAS '98, LNCS 1384, pp. 151–166, doi 10.1007/BFb0054170	Original translation-validation paper springer ↗
McKeeman (1998)	Digital Technical Journal 10(1), 100–107 (no DOI)	Origin of "differential testing" for compilers arxiv ↗ dblp ↗

Pipeline-to-Literature Mapping

Pipeline stage (our prototype)	Primary sources	What we borrow / contrast
MiniArith front end → AST →	Aho et al. (2006)	TAC representation; deliberately

Pipeline stage (our prototype)	Primary sources	What we borrow / contrast
naive TAC		unoptimized lowering
Mutation operators M1–M7, M9	Aho et al. (2006)	Textbook local optimizations recast as single-site rewrites
M8 Instruction Reordering	Schkufza et al. (2013)	Analogue of STOKE's "Swap" move, restricted to independent instructions
Candidate generation + dedup	Petke et al. (2018); Kulkarni et al. (2006)	GI-style mutation; pruning identical candidates
Differential-test validator (25 inputs, output + error status)	Yang et al. (2011); Schkufza et al. (2013)	Differential testing; testcase-based correctness during search
Reference AST interpreter final check	Yang et al. (2011)	Independent implementation as oracle
Weighted cost model + cycle simulator	Schkufza et al. (2013); Petke et al. (2018)	Static latency estimate; multi-objective fitness
Greedy best-improvement search	Kulkarni et al. (2006); Tate et al. (2009)	Phase-ordering problem; equality saturation as the non-greedy contrast
Unsafe MX operator / rejection demo	Lopes et al. (2021); Schkufza et al. (2013)	Shows why per-candidate validation exists
Random-program fuzzing (300 → 1,300 programs)	Yang et al. (2011)	Csmith-style randomized differential testing
Seeded dataset P01–P15	Cummins et al. (2022)	Reproducible benchmark design (at a tiny scale)

Established Research vs. Our Prototype vs. Planned Future Work

Dimension	(1) Established research	(2) Our implemented prototype	(3) Planned future work
IR / scope	Full IRs with control flow	Straight-line MiniArith → TAC;	Branches, loops,

Dimension	(1) Established research	(2) Our implemented prototype	(3) Planned future work
	(LLVM IR, x86-64, Java bytecode)	no branches/loops/CFG	CFG, global data-flow
Transformations	Textbook passes; random opcode/operand moves (STOKE); evolutionary edits (GI)	9 semantics-preserving operators (M1–M9) + unsafe demo MX	More operators (e.g., algebraic identities across blocks)
Search	MCMC (STOKE), GAs (Cooper et al.), exhaustive (Kulkarni et al.), equality saturation (Tate et al.)	Greedy best-improvement, single-site, 500-iteration cap	Simulated annealing, beam search; possibly e-graphs
Correctness	Testcases + SMT validation (STOKE); bounded SMT translation validation (Alive2)	Differential testing on 25 deterministic inputs + reference interpreter; *empirical, not proven*	SMT-based equivalence checking
Performance measure	Static latency models and real runtime (STOKE re-ranks by actual runtime) arxiv ↗	Deterministic simulated cycle model (+,−,<<,copy,io = 1; * = 3; /,% = 10; +1 RAW stall)	Real-hardware timing; 32-bit overflow modelling
Benchmarks	Millions of programs (CompilerGym), cBench, CHStone	15 seeded programs P01–P15, 62 tests	Larger generated sets; possibly real C/LLVM benchmarks

A. Literature Review

Classic optimization foundations. Compiler optimization has traditionally been a fixed sequence of semantics-preserving passes over an intermediate representation such as three-address code. These passes include constant folding and propagation, copy propagation, common-subexpression elimination, dead-code elimination via liveness analysis, and strength reduction (Aho et al., 2006). Each of our operators M1–M9 is one of these rewrites, applied at a single site.

Superoptimization and stochastic search. Superoptimization inverts the pass-based approach by *searching* for the best equivalent program. It began with exhaustive enumeration of short instruction sequences (Massalin, 1987) and was extended to automatically learned peephole rules (Bansal & Aiken, 2006). semanticscholar ↗ researchgate ↗
STOKE turned it into a stochastic search: random mutation moves (opcode, operand, swap, instruction) are scored by a cost combining testcase-measured correctness and estimated performance, and the best candidates are then verified with an SMT solver (Schkufza et al., 2013). arxiv ↗
Our pipeline follows the same propose–test–score structure, but with hand-designed sound rewrites instead of random moves.

Search-based improvement and phase ordering. Genetic improvement generalizes this idea to whole programs: it mutates existing code, uses tests as the correctness oracle, and uses a fitness function for non-functional goals (Petke et al., 2018). semanticscholar ↗
Within compilers, search has been applied to *ordering* optimizations, since phases enable and disable each other. Examples include genetic algorithms for code-size sequences (Cooper et al., 1999), exhaustive enumeration of the phase-order space with pruning of identical results (Kulkarni et al., 2006), and machine-learning autotuning (Ashouri et al., 2018). acm + 2 ↗
Our greedy search implicitly picks one phase order per program, which makes it sensitive to local optima.

Equality saturation. Equality saturation avoids the ordering problem entirely. It records all equivalent forms in a shared e-graph-style IR and extracts the cheapest one at the end (Tate et al., 2009). The egg library made the approach fast and reusable: Willsey et al. (2021) report that, across egg's test suite, congruence is 88× faster and equality saturation 21× faster. It is the principled alternative to our destructive greedy acceptance.

Correctness validation. Translation validation checks each individual compilation result instead of verifying the compiler (Pnueli et al., 1998; Necula, 2000). semanticscholar ↗ berkeley ↗
Alive2 makes this automatic and SMT-based for LLVM IR; sigplan ↗
running it over LLVM's unit test suite found 47 new bugs, 28 of which had already been fixed at publication (Lopes et al., 2021). Differential testing (McKeeman, 1998) and Csmith's random program generation found more than 325 previously unknown bugs in production C compilers, and every compiler tested was found to crash and to silently generate wrong code on valid

input (Yang et al., 2011). These works show that tests find bugs but cannot prove their absence. Our own fuzzing result reproduces that lesson at small scale.

Benchmarks. CompilerGym standardizes optimization research with reproducible environments, more than a million benchmark programs, and built-in differential testing; its authors report it is 27× more computationally efficient than prior works and can detect reproducibility bugs in the underlying compilers (Cummins et al., 2022). This sets the bar for evaluation scale that our seeded 15-program dataset deliberately does not attempt.

B. Research Gap (honest, scoped)

The established tools are powerful but opaque to a student learning compiler design. STOKE operates on x86-64 binaries with MCMC and SMT. Alive2 needs a deep understanding of LLVM semantics. Equality-saturation systems require e-graph machinery. CompilerGym targets reinforcement-learning research on LLVM. arxiv
↗
None of them is designed to show, step by step, *which* textbook optimization fired *where*, *why* it was accepted, and *how* it was validated on a toy IR.

The gap we address is therefore **pedagogical and presentational, not algorithmic**. We provide a fully transparent, deterministic pipeline: named operator-level mutations over TAC, differential testing of every candidate, an explicit weighted cost, and per-candidate logs, on a language small enough to inspect manually.

What our prototype does **not** do (stated explicitly to the panel):

- It does not discover new optimizations. All operators are known transformations.
- It does not prove correctness. Validation is empirical, using 25 inputs plus a reference interpreter.
- It does not handle control flow, memory, or fixed-width overflow.
- It does not measure real hardware performance.
- It does not outperform any production compiler or superoptimizer, and we make no such comparison.

We make **no novelty claim**. The closest prior work is stochastic superoptimization (Schkufza et al., 2013) and genetic improvement (Petke et al., 2018).

C. Research Motivation

1. **Make optimization observable.** Students usually see optimizations as a black-box `-O2` flag. Recasting each textbook rewrite as a mutation operator, and logging every candidate's cost and validation result, makes the effect of each optimization measurable. For example, in our

latest run Copy Propagation (M9) accounted for 32.41% of accepted mutations and Dead Assignment Removal 17.59%.

2. **Demonstrate search-based optimization ideas at small scale.** A greedy propose–validate–score loop shows the core mechanics of STOKE/GI-style search without MCMC or SMT infrastructure. Our latest run evaluated 3,343 candidates with an 8.67% acceptance rate, an average cost reduction of 52.71% (best 89.56% on P06, minimum 15.65% on P12), and an average of 19.33 accepted mutations per program. These figures are measured on *our* simulated cost model only.
3. **Teach the limits of test-based correctness.** In the dataset run, 100% of candidates passed validation (0 rejected), and 62/62 dataset tests plus 4,500/4,500 fresh unseen inputs matched the reference. But fuzzing found that 19 of 300 random programs were optimized incorrectly before the soundness fix, because the 25 validation inputs never triggered division by zero. After the fix (never delete or move a possibly-trapping division across I/O), 1,300 random programs (~20k accepted mutations) showed 0 incorrect results. The lesson matches the translation-validation literature (Necula, 2000; Lopes et al., 2021): operators must be sound on their own, and tests are a safety net, not a guarantee. The 100% validity rate reflects sound operator design. In contrast, STOKE's random moves generate many incorrect candidates and depend on testcases and SMT checks to filter them (Schkufza et al., 2013). arxiv ↗
4. **Provide a clear path to established techniques.** Each limitation maps to a published remedy: SMT-based equivalence checking (Lopes et al., 2021), non-greedy search such as simulated annealing or beam search (Schkufza et al., 2013; Kulkarni et al., 2006), equality saturation (Tate et al., 2009), and realistic benchmarks (Cummins et al., 2022).

D. References (APA 7th, alphabetical)

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2nd ed.). Addison-Wesley.

Ashouri, A. H., Killian, W., Cavazos, J., Palermo, G., & Silvano, C. (2018). A survey on compiler autotuning using machine learning. *ACM Computing Surveys, 51*(5), Article 96, 1–42. https://doi.org/10.1145/3197978

Bansal, S., & Aiken, A. (2006). Automatic generation of peephole superoptimizers. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS XII)* (pp. 394–403). ACM. https://doi.org/10.1145/1168857.1168906

Cooper, K. D., Schielke, P. J., & Subramanian, D. (1999). Optimizing for reduced code space using genetic algorithms. In *Proceedings of the ACM SIGPLAN 1999 Workshop on Languages, Compilers, and Tools for Embedded Systems (LCTES '99)* (pp. 1–9). ACM. https://doi.org/10.1145/314403.314414

Cummins, C., Wasti, B., Guo, J., Cui, B., Ansel, J., Gomez, S., Jain, S., Liu, J., Teytaud, O., Steiner, B., Tian, Y., & Leather, H. (2022). CompilerGym: Robust, performant compiler optimization environments for AI research. In *Proceedings of the 20th IEEE/ACM International Symposium on Code Generation and Optimization (CGO '22)* (pp. 92–105). IEEE. https://doi.org/10.1109/CGO53902.2022.9741258

Kulkarni, P. A., Whalley, D. B., Tyson, G. S., & Davidson, J. W. (2006). Exhaustive optimization phase order space exploration. In *Proceedings of the International Symposium on Code Generation and Optimization (CGO '06)* (pp. 306–318). IEEE. https://doi.org/10.1109/CGO.2006.15

Lopes, N. P., Lee, J., Hur, C.-K., Liu, Z., & Regehr, J. (2021). Alive2: Bounded translation validation for LLVM. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI '21)* (pp. 65–79). ACM. https://doi.org/10.1145/3453483.3454030

Massalin, H. (1987). Superoptimizer: A look at the smallest program. In *Proceedings of the Second International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS II)* (pp. 122–126). ACM. https://doi.org/10.1145/36206.36194

McKeeman, W. M. (1998). Differential testing for software. *Digital Technical Journal, 10*(1), 100–107.

Necula, G. C. (2000). Translation validation for an optimizing compiler. In *Proceedings of the ACM SIGPLAN 2000 Conference on Programming Language Design and Implementation (PLDI '00)* (pp. 83–94). ACM. https://doi.org/10.1145/349299.349314

Petke, J., Haraldsson, S. O., Harman, M., Langdon, W. B., White, D. R., & Woodward, J. R. (2018). Genetic improvement of software: A comprehensive survey. *IEEE Transactions on Evolutionary Computation, 22*(3), 415–432. https://doi.org/10.1109/TEVC.2017.2693219

Pnueli, A., Siegel, M., & Singerman, E. (1998). Translation validation. In B. Steffen (Ed.), *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 1998)* (Lecture Notes in Computer Science, Vol. 1384, pp. 151–166). Springer. https://doi.org/10.1007/BFb0054170

Schkufza, E., Sharma, R., & Aiken, A. (2013). Stochastic superoptimization. In *Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '13)* (pp. 305–316). ACM. https://doi.org/10.1145/2451116.2451150

Tate, R., Stepp, M., Tatlock, Z., & Lerner, S. (2009). Equality saturation: A new approach to optimization. In *Proceedings of the 36th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '09)* (pp. 264–276). ACM. https://doi.org/10.1145/1480881.1480915

Willsey, M., Nandi, C., Wang, Y. R., Flatt, O., Tatlock, Z., & Panchekha, P. (2021). egg: Fast and extensible equality saturation. *Proceedings of the ACM on Programming Languages, 5*(POPL),

Article 23. https://doi.org/10.1145/3434304

Yang, X., Chen, Y., Eide, E., & Regehr, J. (2011). Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '11)* (pp. 283–294). ACM. https://doi.org/10.1145/1993498.1993532

Caveats

- **Verification status:** Every reference above was confirmed through publisher pages (ACM DL, Springer, IEEE via Stirling repository, Pearson), dblp, or arXiv. Two exceptions need care. For Kulkarni et al. (2006), the DOI came from a citing reference and the content summary partly comes from the 2009 TACO follow-up. For the Dragon Book, chapter and page numbers were not checked.
- **STOKE quotes** (32 testcases, STP validator, four moves) come from the arXiv preprint 1211.0557. arxiv ↗
  The final ACM version may differ slightly in wording or section numbers.
- **Some source details apply only to their own settings.** STOKE's results are on x86-64, and CompilerGym's dataset counts are for its LLVM environment. None of these numbers are comparable with our simulated-cycle cost reductions, so do not put them in the same chart.
- **Our numbers** are from the latest internal dataset run and are reported exactly as recorded in the project files. They have not been independently replicated.
- **Souper** (Sasnauskas et al., 2017, arXiv:1711.04422), an SMT-based synthesizing superoptimizer for LLVM IR, arxiv ↗
  arxiv ↗
  was verified but left out to keep the list short. It is a good extra citation if the panel asks about LLVM-level superoptimization.